Dockerfile 寫法

大綱

1. 基本概念
2. 基本指令
3. 建立映像檔
4. 指令詳解
5. 最佳化
6. 多階段建構
7. 實用範例

---

1. 基本概念

1.1 什麼是 Dockerfile?

Dockerfile 是一個文字檔案，包含建立 Docker 映像檔（Image）的指令。透過 Dockerfile，可以自動化地定義容器環境、系統設定與應用程式部署流程。

1.2 為什麼使用 Dockerfile?

- **可重現**：確保環境一致性
- **版本控制**：納入 Git 管理
- **自動化**：擺脫手動設定
- **分享**：方便團隊協作與部署

1.3 基本流程

```
編寫 Dockerfile → docker build → Docker Image → docker run → Container
```

---

2. 基本指令

2.1 常見指令一覽

指令	說明
`FROM`	指定基礎映像檔
`RUN`	執行命令並建立新層
`CMD`	容器啟動時預設命令
`LABEL`	加入標籤 metadata
`EXPOSE`	聲明連接埠

指令	說明
ENV	設定環境變數
ADD	複製檔案（可解壓縮）
COPY	複製檔案
ENTRYPOINT	容器進入點
VOLUME	建立掛載點
USER	設定使用者
WORKDIR	設定工作目錄
ARG	定義建構參數
ONBUILD	延遲執行指令
STOPSIGNAL	停止訊號
HEALTHCHECK	健康檢查
SHELL	設定 shell

2.2 最小範例

```
# 基礎映像檔
FROM nginx:latest

# 複製靜態檔案
COPY ./html /usr/share/nginx/html

# 暴露連接埠
EXPOSE 80

# 啟動指令
CMD ["nginx", "-g", "daemon off;"]
```

---

3. 建立映像檔

3.1 基本指令

```
# 建立映像檔（当前目錄的 Dockerfile）
docker build -t my-app:latest .

# 指定 Dockerfile 路徑
docker build -t my-app:latest -f /path/Dockerfile .

# 使用 build context
docker build -t my-app:latest ./context
```

```
# 帶標籤的建構
docker build -t my-app:latest -t my-app:v1.0.0 .

# 建構時傳遞 ARG
docker build --build-arg VERSION=1.0.0 -t my-app:latest .

# 不使用快取
docker build --no-cache -t my-app:latest .
```

3.2 .dockerignore

在建立上下文根目錄建立 `.dockerignore` 檔案：

```
# 排除版本控制
.git
.gitignore

# 排除相依項目
node_modules
npm-debug.log
__pycache__

# 排除文件
README.md
LICENSE

# 排除建構產出
dist
build
*.log

# 排除 IDE 設定
.vscode
.idea
```

---

4. 指令詳解

4.1 FROM

指定基礎映像檔，必須是 Dockerfile 第一個指令：

```
# 使用官方映像檔
FROM nginx:latest
FROM node:18-alpine
FROM python:3.11-slim

# 使用特定版本
```

```
FROM node:18.17.0-alpine3.18

# 使用別名（多階段建構）
FROM node:18-alpine AS builder
```

4.2 RUN

在建立時執行命令：

```
# Shell 格式
RUN npm install

# Exec 格式（建議）
RUN ["npm", "install"]

# 多行指令
RUN apt-get update && \
    apt-get install -y curl && \
    rm -rf /var/lib/apt/lists/*
```

4.3 CMD

容器啟動時的預設命令：

```
# Shell 格式
CMD echo "Hello"

# Exec 格式（建議）
CMD ["echo", "Hello"]

# 與 ENTRYPOINT 配合
CMD ["world"]
ENTRYPOINT ["echo", "Hello"]

# 覆蓋 CMD
docker run <image> echo "override"
```

4.4 LABEL

加入元資料：

```
LABEL maintainer="user@example.com"
LABEL version="1.0"
LABEL description="My application"
LABEL app.version="1.0.0"
LABEL tier="frontend"
```

4.5 EXPOSE

聲明容器監聽的連接埠：

```
EXPOSE 80
EXPOSE 443
EXPOSE 3000-3010

# 配合 -p 使用
# docker run -p 80:80 <image>
```

4.6 ENV

設定環境變數：

```
# 單個變數
ENV NODE_ENV=production

# 多個變數
ENV APP_HOME=/app \
    DB_HOST=localhost \
    DB_PORT=5432

# 建構 ARG 組合
ARG VERSION
ENV APP_VERSION=$VERSION
```

4.7 ADD 與 COPY

```
# ADD：複製並可處理 URL 與壓縮檔
ADD ./package.json /app/
ADD archive.tar.gz /app/

# COPY：純複製（建議）
COPY ./package.json /app/
COPY ./src /app/src

# 複製並變更擁有者
COPY --chown=1000:1000 ./app /app
```

4.8 WORKDIR

設定工作目錄：

```
WORKDIR /app
WORKDIR /app/src
```

```
WORKDIR /home/user

# 自動建立目錄
WORKDIR /app/uploads
```

4.9 USER

設定執行使用者：

```
# 建立使用者
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

# 切換使用者
USER appuser

# Node.js 常用
USER node
```

4.10 ARG

建構時傳遞的參數：

```
# 定義 ARG
ARG VERSION=1.0
ARG APP_NAME=myapp

# 使用 ARG
RUN echo "Building $APP_NAME v$VERSION"

# 預設值
ARG NODE_ENV=production
```

4.11 ENTRYPOINT

容器進入點，與 CMD 的差異：

```
# ENTRYPOINT：不可被覆蓋（除非使用 --entrypoint）
ENTRYPOINT ["npm", "start"]

# CMD：可被 docker run 參數覆蓋
CMD ["--help"]

# 組合使用
ENTRYPOINT ["npm"]
CMD ["start"]
```

4.12 VOLUME

建立匿名磁碟區：

```
VOLUME /app/data
VOLUME /var/log
VOLUME ["/app/data", "/var/log"]
```

4.13 HEALTHCHECK

健康檢查：

```
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD curl -f http://localhost/ || exit 1

HEALTHCHECK NONE  # 停用健康檢查
```

4.14 ONBUILD

延遲執行指令（作為觸發器）：

```
ONBUILD ADD . /app/src
ONBUILD RUN npm install
```

---

5. 最佳化

5.1 減少映像檔大小

```
# 使用輕量基礎映像檔
FROM node:18-alpine

# 合併 RUN 指令減少層數
RUN apt-get update && \
    apt-get install -y --no-install-recommends curl && \
    rm -rf /var/lib/apt/lists/*

# 清理快取
RUN npm ci --only=production && \
    rm -rf ~/.npm
```

5.2 利用建構快取

```
# 將不常變動的指令放前面
COPY package*.json ./
RUN npm ci
COPY ./src ./src

# 依重要性排序
# 1. 系統套件
# 2. 應用程式依賴
# 3. 程式碼
```

5.3 使用 .dockerignore

```
node_modules
.git
*.log
npm-debug.log
.env
.env.*
```

5.4 最小化層數

```
# 不好：多個 RUN 產生多層
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get install -y vim

# 好：合併為一個 RUN
RUN apt-get update && \
    apt-get install -y curl vim && \
    rm -rf /var/lib/apt/lists/*
```

5.5 使用多階段建構

詳見下一章節。

---

6. 多階段建構

6.1 基本概念

多階段建構允许在一個 Dockerfile 中使用多個 FROM 指令，每個階段可以有不同的基礎映像檔，最終只會複製最後一個階段的產物到最終映像檔。

6.2 Node.js 範例

```dockerfile
# 階段 1: 建構
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# 階段 2: 執行（輕量）
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
CMD ["node", "dist/index.js"]
```

6.3 Go 應用程式範例

```dockerfile
# 階段 1: 建構
FROM golang:1.21-alpine AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

# 階段 2: 執行
FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=builder /app/myapp .
CMD ["./myapp"]
```

6.4 Python 範例

```dockerfile
# 階段 1: 建構
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

# 階段 2: 執行
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /app /app
COPY --from=builder /root/.cache/pip /root/.cache/pip
CMD ["python", "app.py"]
```

6.5 生產環境與開發環境

```
# 階段 1: 建構
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# 階段 2: 生產環境
FROM node:18-alpine AS production
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/package.json .
RUN npm ci --only=production
CMD ["node", "dist/index.js"]

# 階段 3: 開發環境
FROM node:18-alpine AS development
WORKDIR /app
COPY --from=builder /app/package.json .
RUN npm ci
COPY . .
CMD ["npm", "run", "dev"]
```

---

7. 實用範例

7.1 Nginx 靜態網站

```
# 使用 Alpine 版本減少大小
FROM nginx:alpine

# 複製網站檔案
COPY ./html /usr/share/nginx/html

# 複製設定檔（可選）
COPY ./nginx.conf /etc/nginx/nginx.conf

# 暴露連接埠
EXPOSE 80

# 啟動 Nginx
CMD ["nginx", "-g", "daemon off;"]
```

7.2 Node.js API

```dockerfile
# 建構階段
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# 執行階段
FROM node:18-alpine AS production
WORKDIR /app

# 建立使用者
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nodeuser -u 1001

COPY --chown=nodeuser:nodejs --from=builder /app/dist ./dist
COPY --chown=nodeuser:nodejs --from=builder /app/node_modules ./node_modules
COPY --chown=nodeuser:nodejs --from=builder /app/package.json .

USER nodeuser

EXPOSE 3000
CMD ["node", "dist/index.js"]
```

7.3 Python Flask

```dockerfile
FROM python:3.11-slim

WORKDIR /app

# 安裝依賴
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 複製應用程式
COPY app.py .
COPY templates/ ./templates/
COPY static/ ./static/

ENV FLASK_APP=app.py
ENV FLASK_ENV=production

EXPOSE 5000

CMD ["flask", "run", "--host=0.0.0.0"]
```

7.4 Java Spring Boot

```dockerfile
# 建構階段
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

# 執行階段
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app

# 建立非 root 使用者
RUN addgroup -g 1001 app && \
    adduser -u 1001 -G app -D app

COPY --from=builder /app/target/*.jar app.jar

USER app

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

7.5 全面最佳化範例

```dockerfile
# 使用特定版本而非 latest
FROM node:18.17.0-alpine3.18

# 設定維護者（元資料）
LABEL maintainer="dev@example.com"
LABEL version="1.0.0"
LABEL description="Production Node.js application"

# 設定工作目錄
WORKDIR /app

# 先複製依賴檔案（利用建構快取）
COPY package*.json ./

# 安裝依賴
RUN npm ci --only=production && \
    npm cache clean --force

# 複製應用程式碼
COPY --chown=node:node . .

# 設定使用者
USER node
```

```
# 設定環境變數
ENV NODE_ENV=production
ENV PORT=3000

# 暴露連接埠
EXPOSE 3000

# 健康檢查
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD curl -f http://localhost:3000/health || exit 1

# 啟動命令
CMD ["node", "server.js"]
```

總結

- **FROM**：永遠是第一個指令，選擇適當的基礎映像檔
- **最小化**：使用 Alpine、改用 COPY、合併 RUN 指令
- **多階段**：建構與執行分離，大幅減少最終映像檔大小
- **快取利用**：將不常變動的指令放在前面
- **非 root**：建立專用使用者提升安全性
- **健康檢查**：使用 HEALTHCHECK 確保容器正常運行